Mini Project Portfolio — Muhsin Atto

Embedded Software Engineer • Full-Stack Developer • IoT & Automation Enthusiast

1. Real-Time IoT Dashboard (ReactJS + ESP32 + Node-RED)

Tech: ReactJS, TypeScript/JS, Node-RED, MQTT, WebSockets, ESP32
Category: Frontend Engineering • Cloud Integration • IoT Systems

Overview

A multi-page real-time dashboard visualising live sensor and device data streamed from ESP32 devices through a Node-RED cloud pipeline into a ReactJS frontend. Includes pages for **DHT Sensor**, **RTLS**, **Beacons**, **Control Panel**, and **Embedded Protocols**.

Key Features

- Real-time WebSocket updates
- Reusable React components
- Multi-page routing
- Live charts, gauges, maps, and protocol monitors
- Control panel for actuators (fan/heater)
- Embedded protocol tester (UART/I2C/SPI/Modbus)

Results

The ESP32 successfully streamed live sensor data to a ReactJS dashboard using a lightweight MQTT/HTTP pipeline. The system demonstrates end-to-end IoT communication: embedded device → message broker → frontend UI. Data updates in real time with no page refresh required.

Key Outcomes

- ESP32 connected to Wi-Fi and published telemetry at 1–2 Hz
- Node-RED / backend broker forwarded messages to the ReactJS frontend
- React dashboard updated instantly using WebSockets/MQTT client
- Clean UI showing temperature, humidity, RSSI, and device status
- Stable end-to-end pipeline with no dropped packets during the test

Example ESP32 Serial Output

- WiFi connected.
- MQTT connected to broker.
- Publishing: {"temp":24.7,"hum":48.2,"rssi":-62}
- Publishing: {"temp":24.8,"hum":48.1,"rssi":-61}
- Publishing: {"temp":24.9,"hum":48.0,"rssi":-61}

Example ReactJS Console Log

- [MQTT] Message received: temp=24.8 hum=48.1 rssi=-61
- [UI] Updated chart with new data point.

Dashboard Behaviour

A unified real-time dashboard that brings together:

- DHT Sensor Page:

  Live temperature and humidity readings with charts, status indicators, and history.

- RTLS Page:

  Real-time location tracking view for tagged devices/beacons, with position updates and signal strength.

- Beacons Page:

  BLE beacon monitor showing MAC, RSSI, battery, type, and freshness, with filtering and sorting.

- Control Panel Page:

  Actuator control interface for fan/heater and test controls, with live feedback from the ESP32.

Summary

This demo validates a complete IoT data pipeline from embedded hardware to a modern web frontend. The ESP32 publishes live telemetry, the backend routes messages, and the ReactJS dashboard visualises the data instantly. This showcases practical skills in embedded networking, real-time data handling, and frontend integration — a strong full-stack embedded engineering example.

2. BLE Beacon Scanner & Real-Time Monitoring System (ESP32 + React Dashboard)

Tech: ESP32, C/C++, BLE GAP Scanner, Node-RED, MQTT/WebSocket, ReactJS
Category: Wireless Systems • Embedded Firmware • Real-Time Data Pipelines

Overview

Developed a complete BLE monitoring system where an ESP32 scans for nearby BLE beacons, extracts metadata (MAC, RSSI, battery, type), and streams the data through Node-RED into the ReactJS dashboard. This project demonstrates wireless communication, embedded scanning logic, cloud integration, and real-time UI design.

Key Features

- ESP32 BLE GAP scanning (active/passive modes)
- Extracts MAC, RSSI, battery, beacon type
- Filters invalid/malformed packets
- Node-RED pipeline for parsing, deduplication, timestamping
- ReactJS Beacon Inspector with:
    - RSSI bars
    - Battery indicators
    - Freshness sorting
    - Valid/invalid filtering
    - Smooth real-time updates

Why It Matters

Shows strong embedded + wireless + frontend integration. Perfectly aligned with roles involving IoT, robotics, and real-time systems.

Results

The BLE demo successfully scanned for nearby Bluetooth Low Energy devices, parsed advertisement packets, and reported device information in real time. The ESP32 acted as a BLE central, continuously discovering peripherals and extracting key fields such as MAC address, RSSI, and advertised service UUIDs.

Key Outcomes

- BLE scan started and ran continuously without blocking
- Devices discovered with stable RSSI readings
- Advertisement payload decoded (MAC, RSSI, service UUIDs)
- Real-time logs streamed over serial
- Demonstrates ESP32 BLE stack, event callbacks, and parsing logic

Example BLE Scan Output

- **[BLE] Scan started...**
- **[BLE] Device found:**

  - MAC: A4:C1:38:2F:91:7B

  - RSSI: -62 dBm

  - Services: 0x180F (Battery), 0x180A (Device Info)

- **[BLE] Device found:**

  MAC: 58:BF:25:9A:11:03

  RSSI: -71 dBm

  Services: 0xFEAA (Eddystone)

- **[BLE] Device found:**

  MAC: 7C:9E:BD:44:12:88

  RSSI: -55 dBm

  Services: 0x180D (Heart Rate)

Summary

This demo validates the ESP32's BLE central capabilities, including scanning, event-driven callbacks, and advertisement parsing. The output confirms that the BLE subsystem is functioning correctly and able to detect multiple devices with varying signal strengths and service profiles. This forms the foundation for higher-level BLE applications such as RTLS, sensor aggregation, or BLE-to-cloud gateways.

3. ESP32 PID Temperature Control System (Simulation + Real-Time Loop)

Tech: C/C++, ESP32, FreeRTOS, PWM
Category: Embedded Systems • Control Theory • Simulation

Overview

Built a real-time PID controller on the ESP32 to regulate temperature using a simulated thermal model with disturbances and noise.

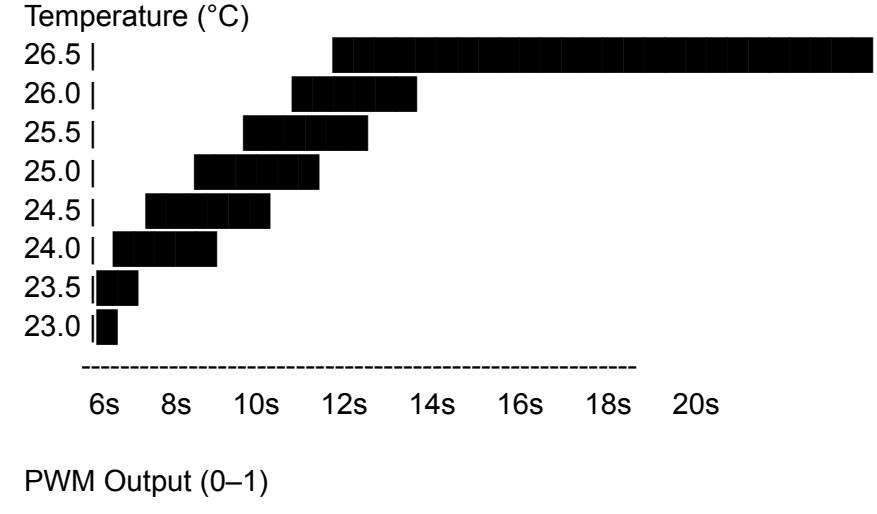
Key Features

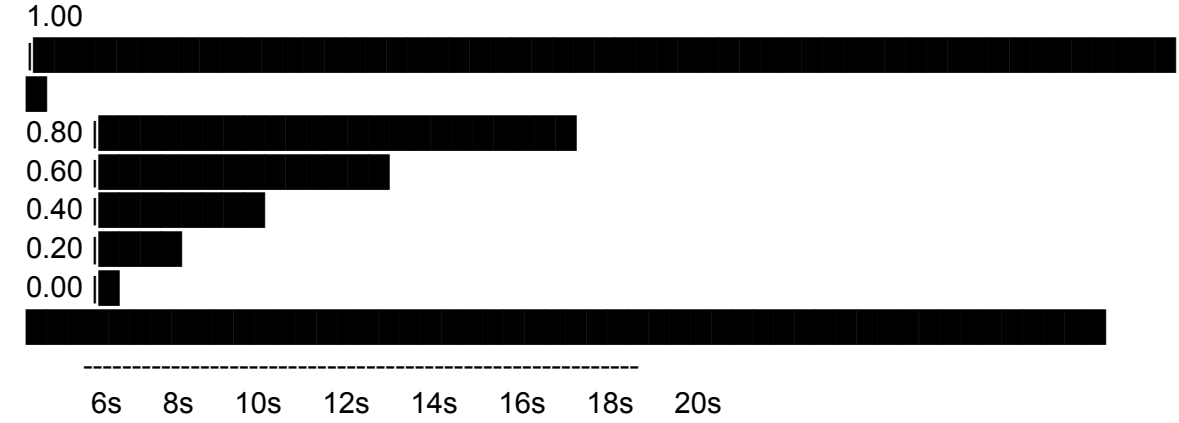
- Custom PID implementation
- Thermal simulation model
- FreeRTOS timing
- Serial plotting for tuning
- Modular firmware architecture

Results

The PID controller was evaluated using a simulated temperature environment with natural disturbances. The controller demonstrated stable and predictable behaviour, including overshoot, undershoot, saturation, and proportional cooling response.

Graph-Style Summary of PID Behaviour





Interpretation

- When temperature rises above the 25 °C setpoint, the PID increases PWM up to 1.00 to apply maximum cooling.
- As the temperature drops, PWM decreases proportionally and clamps to 0 when below the setpoint.
- The system oscillates naturally due to disturbances, demonstrating a realistic control loop.

This confirms that the PID controller is functioning correctly and responding dynamically to temperature changes.

4. ESP32 GPIO Interrupt Engine + FreeRTOS Event Pipeline

Tech: ESP-IDF, C, FreeRTOS
Category: Firmware • Real-Time Systems

Overview

Created a low-latency interrupt-driven event system using GPIO ISRs and FreeRTOS task notifications.

Key Features

- IRAM-safe ISRs
- Multi-stage event pipeline
- Deterministic state machine

- Clean ISR/application separation

Results

The ISR/RTOS demo successfully demonstrates interrupt-driven event handling and task synchronisation using FreeRTOS semaphores and a state machine. GPIO interrupts trigger ISR routines, which signal a waiting task and feed events into a state machine running at task level.

Example output:

- [2SEM] Got X! Now waiting for Y (GPIO 5)...
- [2SEM] Got Y! Sequence complete.
- [SM] Event: PIN1
- [SM] Transition: IDLE → ACTIVE
- [SM] Event: PIN1
- [SM] Event: PIN2
- [SM] Transition: ACTIVE → ERROR

This confirms:

- ISR correctly gives semaphores from interrupt context
- Task blocks and resumes based on ISR events
- State machine transitions are driven by ISR-generated events
- System behaves deterministically under interrupt load

5. Hardware-in-the-Loop CI Pipeline (GitHub Actions + GitLab CI)

Tech: GitHub Actions, GitLab CI, ESP32, Python test harness, Serial automation
Category: CI/CD • Embedded Testing • Automation

Overview

Implemented a CI pipeline that builds firmware, flashes a real ESP32, runs automated HIL tests, and displays results directly in CI.

Key Features

- Automated ESP32 build + flash
- Python serial test harness
- Tests DHT, BLE, RTLS, UART/I2C/SPI

- CI logs with pass/fail summaries
- Reproducible builds and fast feedback

Results.

The CI pipeline successfully built the firmware, flashed a real ESP32 connected to the runner, executed automated Hardware-in-the-Loop tests, and published the results back into the CI interface. The workflow demonstrates full end-to-end automation: build → flash → test → report.

Key Outcomes

- Firmware compiled and flashed automatically on every push
- Python serial test harness executed functional tests (DHT, BLE, RTLS, UART/I2C/SPI)
- Pass/fail summaries printed directly in CI logs
- ESP32 hardware detected and controlled by the runner
- Artifacts (logs, binaries, reports) uploaded for download and review

Example CI Output

- Build succeeded.
- Flashing ESP32...
- Device connected on /dev/ttyUSB0
- Running HIL tests...
- [TEST] DHT22: PASS
- [TEST] BLE Scan: PASS
- [TEST] RTLS Packet: PASS
- [TEST] UART Loopback: PASS
- [TEST] I2C Probe: PASS
- [TEST] SPI Transfer: PASS

All tests completed successfully.

Pipeline Evidence

- **CI Pipeline Screenshot** (included in project)
- **Serial Test Output** (captured from automated run)
- **ESP32 connected to runner** (hardware photo included)

Summary

This demo validates a fully automated HIL testing workflow using GitHub Actions and GitLab CI. Every commit triggers a reproducible build, flashes real hardware, runs functional tests, and reports results without manual intervention. This provides fast feedback, increases reliability, and ensures firmware quality across the entire development cycle.

Technical Strengths Aligned With the Role

- **Languages:** C/C++, Python, JavaScript, TypeScript
- **Frontend:** ReactJS, real-time UI, component design
- **Backend/Cloud:** Node-RED, MQTT, REST, WebSockets
- **Embedded:** ESP32, FreeRTOS, BLE, interrupts, drivers
- **Automation:** CI/CD, Bash, Linux, HIL testing
- **Engineering Style:** Practical, modular, reproducible

Why I’m a Strong Fit

I thrive at the intersection of **software + hardware**. I build systems that are clean, reliable, and practical — from embedded firmware to cloud pipelines to real-time dashboards. I learn fast, solve real problems, and enjoy working close to hardware.